

Vibe Coding从入门到精通

项目宪章

AI Agent开发平台每次对话开始时会自动读取项目宪章.md。这个文件不是说明书，它更像一份契约。你和AI之间关于怎么干活的约定，就写在这一个文件里。

每个开发AI开发工具里名称不一致，但一定存在，在CodeBuddy里面名称为 `CODEBUDDY.md`，在CLAUDE Code里文件名称为CLAUDE.md

为什么它是最重要的文件

用vibe Coding写代码，你会接触到很多配置文件。package.json、tsconfig.json、.eslintrc ...等等这些项目的配置文件，但有一个文件的重要性超过它们加起来。

项目宪章文件：`CODEBUDDY.md`

原因很简单：AI Agent每次启动新会话，第一件事就是读这个文件。项目结构、代码风格、测试命令、常见陷阱，AI都从这里了解。没有它，AI就像空降到陌生代码库的新同事，什么都得从头摸索。有了它，它一进来就知道规矩。

Shrivu Shankar (Abnormal AI的AI战略VP，团队每月消耗数十亿tokens做代码生成) 说得很直白：

在有效使用vibe Coding时，代码库中最重要的文件就是根目录的CLAUDE.md。这个文件是agent的「宪法」，是它了解你的特定代码库如何工

他用了「宪法」这个词。宪法的特点是短、原则性强、不处理细节。这个类比很精确。

从护栏开始，别写手册

新手写项目宪章.md最常犯的错：试图写一本百科全书。把每个函数的用法、每个文件的作用、每个API的参数都塞进去。写了几千行，AI光读这个文件就吃掉大量上下文，真正干活的空间反而被挤小了。

Boris (Claude Code的创建者) 团队的CLAUDE.md只有大约2500 tokens，大概100行。管理Claude Code这产品本身的核心规则文件，就这么短。

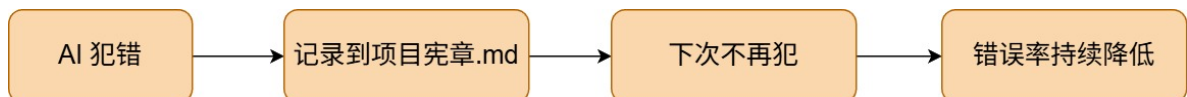
Shrivu分享了一个更有意思的做法：

核心建议

你的项目宪章.md应该从小开始，基于AI做错的事情来记录。不要试图预先写一本完整手册，而是每次AI犯错，就加一条规则。这就是「从护栏开始」的意思。

这个方法好在哪？规则文件永远精准，因为每条都对应一个真实踩过的坑。文件也天然保持精简，因为你只记录真正出过问题的地方。

Boris在他的使用技巧中还提到一个飞轮效应：



团队共享一个项目宪法.md文件，check进git，每周都有人贡献。这个文件是活的，不是写完就放那不管的。

项目宪法.md到底该写什么

这可能是最实用的部分。判断标准就一条：AI Agent自己能从代码里读出来的，不要写；AI猜不到的，必须写。

该写	不该写
猜不到的Bash命令(如自定义构建脚本)	读代码就能知道的事(如「这是一个React项目」)
与默认不同的代码风格偏好	标准语言规范(AI已经知道)
测试命令和偏好的测试框架	详细API文档(给链接，不要全文粘贴)
项目架构决策和背景	频繁变化的信息(每次都要改的东西不适合放这里)
开发环境的坑(如特殊的环境变量)	文件逐一描述(AI会自己看文件树)
常见陷阱和修复方式	「写整洁代码」「遵循最佳实践」这种废话

我补充了几个常见反模式：

不要用 @ 引用大文档。在项目宪法.md里 @ 一个长文件，它会在每次会话开始时被完整嵌入，白白吃掉上下文。

正确做法是提到路径，告诉AI什么情况下去读。比如：「遇到FooBarError时，参阅 docs/troubleshooting.md 了解故障排除步骤。」

不要只写「永远不要做X」。当AI觉得必须做X时它会卡住。永远提供替代方案：「不要用 -foo-bar 标志，改用 -baz。」

把项目宪章.md当作简化代码库的强制函数。如果某个CLI命令复杂到需要在项目宪章.md里写几段话来解释，那说明这个命令本身需要简化。写一个bash包装器，用清晰的API，然后在项目宪章.md里只记录那个包装器。

层级结构

项目宪法.md不只是一个文件，而是一套层级系统。AI Agent会自动按顺序读取多个位置的md（每个工具各不相同，根据工具文档去查找）：

下面是codebuddy的项目宪章记忆结构

CodeBuddy Code 提供四种分层结构的记忆位置，每种都有不同的用途：

记忆类型	位置	用途	使用场景示例	共享范围
用户记忆	~/ .codebuddy/CODEBUDDY.md	适用于所有项目的个人偏好	代码风格偏好、个人工具快捷方式	仅限本人（所有项目）
用户规则	~/ .codebuddy/rules/*.md	模块化的个人规则	个人编码习惯、常用工作流	仅限本人（所有项目）
项目记忆	./CODEBUDDY.md 或 ./ .codebuddy/CODEBUDDY.md	项目的团队共享指令	项目架构、编码标准、常用工作流程	通过源代码管理与团队成员共享
项目规则	./ .codebuddy/rules/*.md	模块化的、按主题划分的项目指令	语言特定指南、测试规范、API 标准	通过源代码管理与团队成员共享
项目记忆（本地）	./CODEBUDDY.local.md	个人的项目特定偏好	您的沙箱 URL、首选测试数据	仅限本人（当前项目）

所有记忆文件在启动 CodeBuddy Code 时自动加载到上下文中。加载顺序如下：

- 用户级：**加载 ~/ .codebuddy/CODEBUDDY.md 等主文件及 ~/ .codebuddy/rules/ 下的所有规则
- 项目级主文件：**从当前工作目录向上递归加载所有 CODEBUDDY.md 和 CODEBUDDY.local.md
- 项目级规则：**仅加载当前工作目录的 .codebuddy/rules/ 下的规则（不加载父目录的规则）

- 4. **子目录记忆**：当 CodeBuddy 操作子目录中的文件时，动态加载该子目录的 CODEBUDDY.md
- 5. **本地记忆**：加载 ./CODEBUDDY.local.md

提示：CODEBUDDY.local.md 文件会自动添加到 .gitignore，非常适合存储不应提交到版本控制的私有项目特定偏好。

一个真实的好项目宪法.md长什么样

下面是一个简洁精炼的项目级.md示例。注意它有多短：

```
# MyApp
# 架构
- Next.js 15 + TypeScript + Tailwind CSS
- 数据库: PostgreSQL + Drizzle ORM
- 认证: Better Auth
- 状态管理: Zustand (不要用Redux)
# 开发命令
- 启动开发服务器: pnpm dev
- 跑测试: pnpm test (Jest + React Testing Library)
- 类型检查: pnpm typecheck
- Lint: pnpm lint
# 代码风格
- 组件用函数式，不用class
- 样式用Tailwind，不要写CSS文件
- 数据获取用server component，不用useEffect
- 错误处理用error.tsx边界，不用try-catch包裹组件
# 常见陷阱
- Drizzle迁移后必须跑 pnpm db:generate，否则类型不同步
- 环境变量改了之后要重启dev server
- better-auth的session检查在middleware中，不要在页面组件里重复检查
# 不要做
- 不要安装新依赖除非我明确同意
- 不要修改 drizzle.config.ts
- 不要在client component中直接调数据库
```

整个文件不到300字。但每一行都有价值：要么是Claude猜不到的命令，要么是踩过坑的经验。没有一句废话。

注意

不要把这个示例直接复制过去。好的CLAUDE.md是从你自己的项目中「长出来」的。空文件开始，vibe coding时AI犯一次错就加一条，实训后那个文件就是你的定制护栏。

Auto Memory: AI自己记住的东西

除了你手写的项目宪法.md，AI Agent还有一个自动记忆系统。

当你在对话中纠正AI的行为，比如「以后commit message都用英文」「测试文件放在_tests_目录」，AI会自动把这些偏好保存下来。下次对话它就记住了，不需要你再说一遍，也不需要你手动写进项目宪法.md。

MEMORY.md 文件各自AI开发平台不同，一般不用自己维护

手写CLAUDE .md	Auto Memory
适合团队共享的规则	适合个人偏好
检入git	存在本地

手写CLAUDE.md 你主动维护	Auto Memory Claude自动维护
结构化、有组织	零散、按时间累积

两者配合使用效果最好。团队规则写在项目宪章.md里，个人习惯让Auto Memory自动处理。

迭代飞轮：越用越好的系统

回到开头说的飞轮。这不只是个比喻，它就是Vibe coding用户的真实体验曲线。

这个过程是这样的：

1，第一周：空文件

你只写了基本的项目架构和开发命令。AI犯很多错。

2，第二周：护栏初现

你把AI犯过的错一条条记下来。「不要在这个文件里用相对路径」「跑完迁移记得重新生成类型」。错误率开始下降。

3，第一个月：飞轮启动

项目宪章.md有了20-30条规则，都是真实的坑。AI的输出质量明显提升，你需要纠正的次数越来越少。

4，之后：持续迭代

偶尔加新规则，偶尔删掉过时的。文件保持精简但高度定制。你把同样的方法迁移到新项目，启动速度越来越快。

这就是为什么说项目宪章.md是最重要的文件。每一条规则背后都是一次真实踩过的坑，每次迭代都让AI更懂你的项目。